

MI RGN API

Version 3.0

© 2022 SigmaStar Technology Corp. All rights reserved.

SigmaStar Technology makes no representations or warranties including, for example but not limited to, warranties of merchantability, fitness for a particular purpose, non-infringement of any intellectual property right or the accuracy or completeness of this document, and reserves the right to make changes without further notice to any products herein to improve reliability, function or design. No responsibility is assumed by SigmaStar Technology arising out of the application or use of any product or circuit described herein; neither does it convey any license under its patent rights, nor the rights of others.

SigmaStar is a trademark of SigmaStar Technology Corp. Other trademarks or names herein are only for identification purposes only and owned by their respective owners.

REVISION HISTORY

Revision No.	Description	Date
3.0	• Initial release	12/04/2020
	• Added PROCFS introduction	08/25/2021
	• Added chip information of Mochi	02/16/2022
	• Added chip information of Maruko	03/21/2022

TABLE OF CONTENTS

REVISION HISTORY	i
TABLE OF CONTENTS	ii
1. 概述	1
1.1. 模块说明	1
1.2. 流程框图	2
1.3. 关键字说明	2
1.4. RGN 模块内存使用说明	2
2. API 参考	4
2.1. API List	4
2.1.1 MI_RGN_Init	4
2.1.2 MI_RGN_DeInit	5
2.1.3 MI_RGN_Create	6
2.1.4 MI_RGN_Destroy	8
2.1.5 MI_RGN_GetAttr	8
2.1.6 MI_RGN_SetBitMap	9
2.1.7 MI_RGN_AttachToChn	11
2.1.8 MI_RGN_DetachFromChn	15
2.1.9 MI_RGN_SetDisplayAttr	15
2.1.10 MI_RGN_GetDisplayAttr	17
2.1.11 MI_RGN_GetCanvasInfo	18
2.1.12 MI_RGN_UpdateCanvas	20
2.1.13 MI_RGN_InitDev	21
2.1.14 MI_RGN_DeInitDev	21
3. RGN 数据类型	23
3.1. MI_RGN_MAX_HANDLE	24
3.2. MI_RGN_MAX_PALETTE_TABLE_NUM	24
3.3. MI_RGN_HANDLE	24
3.4. MI_RGN_Type_e	25
3.5. MI_RGN_PixelFormat_e	25
3.6. MI_RGN_AlphaMode_e	26
3.7. MI_RGN_Size_t	27
3.8. MI_RGN_OsdAlphaAttr_t	27
3.9. MI_RGN_OsdInitParam_t	28
3.10. MI_RGN_PaletteElement_t	29
3.11. MI_RGN_PaletteTable_t	29
3.12. MI_RGN_Attr_t	30
3.13. MI_RGN_Bitmap_t	30
3.14. MI_RGN_ChnPort_t	31
3.15. MI_RGN_Point_t	31
3.16. MI_RGN_CoverChnPortParam_t	32
3.17. MI_RGN_OsdChnPortParam_t	33
3.18. MI_RGN_OsdArgb1555Alpha_t	33
3.19. MI_RGN_ChnPortParamUnion_u	34

3.20. MI_RGN_AlphaModePara_u.....34

3.21. MI_RGN_ChnPortParam_t.....35

3.22. MI_RGN_CanvasInfo_t.....36

3.23. MI_RGN_InitParam_t.....36

4. 返回值.....38

5. PROCFS 介绍.....39

5.1. cat.....39

5.2. echo.....42

1. 概述

1.1. 模块说明

区域管理模块参与 SCL 模块的内部流处理的一个环节。底层硬件模块支持是 GOP（Graphic output path），区域管理模块是利用 GOP 的特性抽象出来的一套软件接口，利用分时复用的原理使 OSD(On-screen display)或者 Cover 贴到各个通道上。

区域管理模块提供区域资源的控制管理功能，包括区域的创建、销毁、获取与设置区域属性、获取与设置区域的通道属性等。

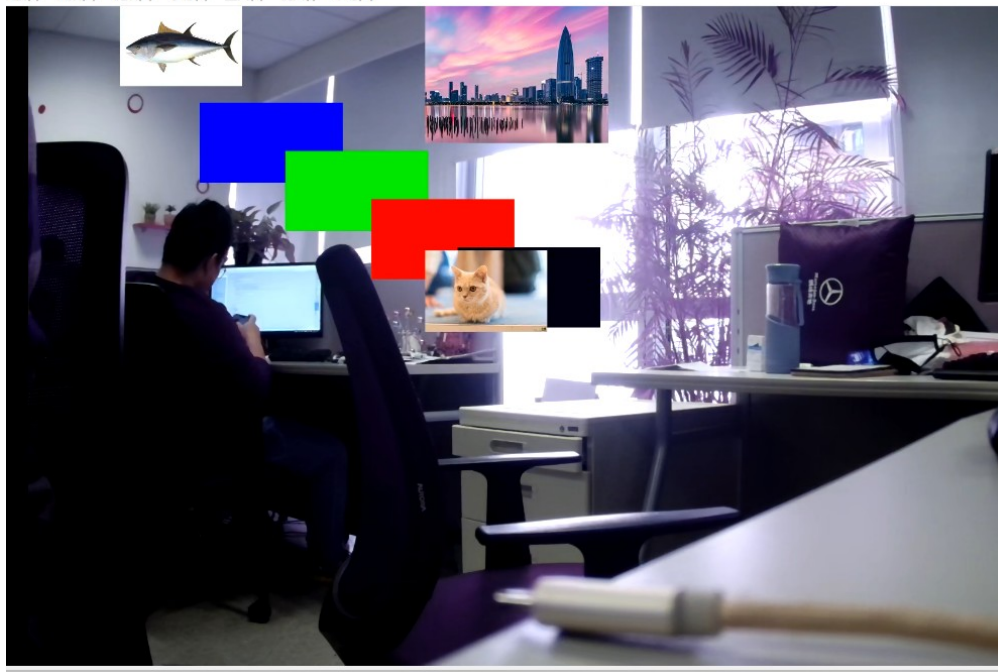
区域属性分为两种，一种是 **cover**，**cover** 只是按照设定，给一块区域做遮挡，底层驱动只要知道显示的位置大小和颜色即可，因此创建 **cover** 时，不会给 **cover** 分配内存做显示。另外一种区域属性称为 **osd**，有些地方亦称为 **overlay**，它同样可以贴在 **video** 显示的区域，这块区域用内存来描述显示的内容，所以这块内容可以做点对点画图操作，显示内存支持 **argb**、位图两种格式，使用者可以根据实际的场景来选择所需要的格式。

目前 OSD 可以支持的格式有 **ARGB1555**、**ARGB4444**、**ARGB8888**、**RGB565**、**I2**、**I4**、**I8**，每个芯片支持的情况会有差异，下文会详细说明。暂不支持 **YUV** 格式。**I2**、**I4**、**I8** 格式为位图格式，一个 **pixel** 的内存数据当作一个索引，通过索引能找到调色盘中的颜色数据，当前 **pixel** 显示的就是此颜色。

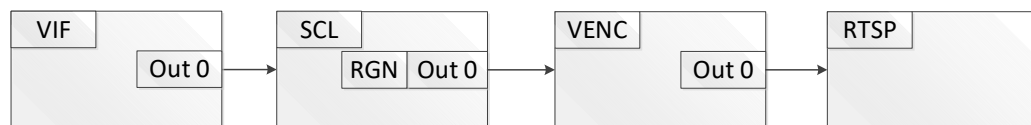
无论是 **cover** 还是 **OSD** 其显示的内容均是叠加到 **SCL** 模块的输出上，通过实验，当设定了 **region** 显示之后，把 **SCL** 模块的输出 **dump** 成文件，通过工具就可以看到 **region** 的内容。

OSD 可以显示一张图片，**COVER** 仅仅可以设定颜色，在同一通道上 **OSD** 的内容永远在 **COVER** 之上，下图在同一个通道上显示了四个 **cover** 区域以及三张 **OSD**。

播放(L) 音频(A) 视频(V) 字幕(T) 工具(S) 视图(I) 帮助(H)



1.2. 流程框图



1.3. 关键字说明

- **I2**
4 色位图格式，用 2 个 bit 表示一个索引，因而有 4 种颜色，在调色盘中通过索引找到对应的颜色。
- **I4**
16 色位图格式，与 I2 格式类似，不同的是用 4 个 bit 表示一个索引，因而有 16 种颜色。
- **I8**
256 色位图格式，与 I2 格式类似，不同的是用 8 个 bit 表示一个索引，因而有 256 种颜色。
- **Palette**
位图的调色盘，由 Alpha、Red、Green、Blue 四个 8bit 变量表示一个 pixel 的颜色，一共有 256 个颜色，对应 0-255 的索引序号。
- **OSD**
On-screen display 的简称，用来显示一些文字、图片、以及人机交互的菜单等内容。
- **GOP**
Graphic output path 的简称，可以理解为在 video 层之上的一个图形层。

1.4. RGN 模块内存使用说明

为确保 OSD 能稳定输出在 video 上，RGN 根据硬件/用户的使用情况为每个 OSD 分配一到多张 buffer，中心思想为防止对正在用于硬件显示的 buffer 做写操作，从而造成 OSD 闪烁或撕裂的现象。

可能会增加 RGN 内存占用的情况主要有以下 3 种：

1. 上层刷新 OSD（调用 MI_RGN_SetBitMap/MI_RGN_GetCanvasInfo/MI_RGN_UpdateCanvas）的速度变快；
2. 一个 OSD 被同时叠加到的通道数增多；
3. 一个通道同时叠加的 OSD 数超过硬件最大 layer（每个芯片支持的硬件最大 layer 数见表 2 -1: Tiramisu 芯片信息和表 2 -2: Muffin 芯片信息）。

下表列出了几种常见的场景下 buffer 使用情况：

场景描述	OSD 个数	OSD 刷新速度 t/次	通道个数	buffer 使用
1 个 OSD 贴到 1 个通道，只贴一次不刷新	1	NA	1	1
1 个 OSD 贴到 1 个通道，慢速刷新	1	1s	1	2
1 个 OSD 贴到 1 个通道，快速刷新	1	30ms	1	3
1 个 OSD 贴到 n 个通道，只贴一次不刷新	1	NA	n	1
1 个 OSD 贴到 n 个通道，慢速刷新	1	1s	n	2

场景描述	OSD 个数	OSD 刷新速度 t/次	通道个数	buffer 使用
1 个 OSD 贴到 n 个通道，快速刷新	1	30ms	n	$[3, 2n+1]$
m 个 OSD 贴到 1 个通道，只贴一次不刷新	$m \leq L$	NA	1	m
m 个 OSD 贴到 1 个通道，慢速刷新	$m \leq L$	1s	1	2m
m 个 OSD 贴到 1 个通道，快速刷新	$m \leq L$	30ms	1	3m
m 个 OSD 贴到 n 个通道，只贴一次不刷新	$m \leq L$	NA	n	m
m 个 OSD 贴到 n 个通道，慢速刷新	$m \leq L$	1s	n	2m
m 个 OSD 贴到 n 个通道，快速刷新	$m \leq L$	30ms	n	$[3m, m(2n+1)]$
m 个 OSD 贴到 n 个通道，只贴一次不刷新	$m > L$	NA	n	$L + n(m - L)$
m 个 OSD 贴到 n 个通道，慢速刷新	$m > L$	1s	n	$2L + 2n(m - L)$
m 个 OSD 贴到 n 个通道，快速刷新	$m > L$	30ms	n	$[3L, L(2n+1)] + 3n(m-L)$

注：

- 1: L 表示硬件最大 layer 数；
- 2: 形如 $[3, 2n+1]$ 表示闭区间，buffer 个数为该区间内的某个值（受系统调度影响，通常不会到达最大值）。

2. API 参考

2.1. API List

API 名	功能
MI_RGN_Init	初始化
MI_RGN_DeInit	反初始化
MI_RGN_Create	创建区域
MI_RGN_Destroy	销毁区域
MI_RGN_GetAttr	获取区域属性
MI_RGN_SetBitMap	设置区域位图
MI_RGN_AttachToChn	将区域叠加到通道上
MI_RGN_DetachFromChn	将区域从通道中撤出
MI_RGN_SetDisplayAttr	设置区域的通道显示属性
MI_RGN_GetDisplayAttr	获取区域的通道显示属性
MI_RGN_GetCanvasInfo	获取区域画布信息
MI_RGN_UpdateCanvas	更新区域画布信息
MI_RGN_InitDev	初始化 rgn 设备
MI_RGN_DeInitDev	反初始化 rgn 设备

2.1.1 MI_RGN_Init

➤ 功能

初始化。

➤ 语法

MI_S32 MI_RGN_Init(MI_U16 u16SocId, [MI_RGN_PaletteTable_t](#) *pstPaletteTable);

➤ 形参

参数名称	参数含义	输入/输出
s32SocId	芯片 ID，用于级联场景。	输入
pstPaletteTable	调色板指针。	输入

➤ 返回值

返回值 { [MI_RGN_OK](#) 成功。
[MI_ERR_RGN_BUSY](#) 已初始化。

➤ 依赖

- 头文件：mi_sys.h、mi_rgn.h。
- 库文件

※ 注意

- I2/I4/I8 的图像格式是共用一份 palette table，palette table 只能在初始化时做一次，不可再次设置。
- RGB 格式不会参考 palette。
- Palette 的第 0 号成员为透明色，上层无法指定。
- MI_RGN_Init 需要在其他使用到 RGN 的模块 Init 之前进行（如 SCL）。

➤ 举例

```
MI_S32 s32Result = 0;
MI_RGN_PaletteTable_t stPaletteTable;
memset(&stPaletteTable, 0, sizeof(MI_RGN_PaletteTable_t));
stPaletteTable.astElement[1].u8Alpha = 255;
stPaletteTable.astElement[1].u8Red = 255;
stPaletteTable.astElement[1].u8Green = 0;
stPaletteTable.astElement[1].u8Blue = 0;
stPaletteTable.astElement[2].u8Alpha = 255;
stPaletteTable.astElement[2].u8Red = 0;
stPaletteTable.astElement[2].u8Green = 255;
stPaletteTable.astElement[2].u8Blue = 0;
stPaletteTable.astElement[3].u8Alpha = 255;
stPaletteTable.astElement[3].u8Red = 0;
stPaletteTable.astElement[3].u8Green = 0;
stPaletteTable.astElement[3].u8Blue = 255;
s32Result = MI_RGN_Init(0, &stPaletteTable);
s32Result = MI_RGN_DeInit(0);
```

➤ 相关主题

[MI_RGN_DeInit](#)

2.1.2 MI_RGN_DeInit

➤ 功能

反初始化。

➤ 语法

```
MI_S32 MI_RGN_DeInit(MI_U16 u16SocId);
```

➤ 形参

参数名称	参数含义	输入/输出
s32SocId	芯片 ID，用于级联场景。	输入

➤ 返回值

返回值 { [MI_RGN_OK](#) 成功。
[MI_ERR_RGN_BUSY](#) 未初始化。

➤ 依赖

- 头文件：mi_sys.h、mi_rgn.h。
- 库文件

※ 注意

- MI_RGN_DeInit 需要在其他使用到 RGN 的模块 Deinit 之后进行（如 SCL）。

➤ 举例

参见 [MI_RGN_Init](#) 举例。

➤ 相关主题

[MI_RGN_Init](#)

2.1.3 MI_RGN_Create

➤ 功能

创建区域。

➤ 语法

```
MI_S32 MI_RGN_Create(MI_U16 u16SocId, MI\_RGN\_HANDLE hHandle,  
MI\_RGN\_Attr\_t *pstRegion);
```

➤ 形参

参数名称	参数含义	输入/输出
s32SocId	芯片 ID，用于级联场景。	输入
hHandle	区域句柄号。 必须是未使用的 hHandle 号 取值范围：[0, MI_RGN_MAX_HANDLE]。	输入
pstRegion	区域属性指针。	输入

➤ 返回值

{ [MI_RGN_OK](#) 成功。

返回值

非 [MI_RGN_OK](#) 失败，参照[返回值](#)。

➤ 依赖

- 头文件：mi_sys.h、mi_rgn.h。
- 库文件

※ 注意

- 该句柄由用户指定，意义等同于 ID 号。
- 不支持重复创建。
- 区域属性必须合法，具体约束参见 [MI_RGN_Attr_t](#)。
- [MI_RGN_Attr_t](#) 中指定 Cover 还是 OSD
- 区域属性指针不能为空。
- 创建 Cover 时，只需指定区域类型即可。其它的属性，如区域位置，层次等信息在调用 [MI_RGN_AttachToChn](#) 接口时指定。
- 创建区域时，本接口只进行基本的参数的检查，例如：最小宽高，最大宽高等；当区域 attach 到通道上时，根据各通道模块支持类型的约束条件进行更加有针对性的参数检查，譬如支持的像素格式等；

➤ 举例

```
MI_S32 s32Result = 0;
MI_RGN_HANDLE hHandle = 0;
MI_RGN_Attr_t stRegion;
stRegion.eType = E_MI_RGN_TYPE_OSD;
stRegion.stOsdInitParam.ePixelFormat = E_MI_RGN_PIXEL_FORMAT_RGB1555;
stRegion.stOsdInitParam.stSize.u32Width = 40;
stRegion.stOsdInitParam.stSize.u32Height = 40;

s32Result = MI_RGN_Create(0, hHandle, &stRegion);
if (s32Result != MI_RGN_OK)
{
    return s32Result;
}

s32Result = MI_RGN_GetAttr(0, hHandle, &stRegion);
if (s32Result != MI_RGN_OK)
{
    return s32Result;
}

s32Result = MI_RGN_Destroy(0, hHandle);
if (s32Result != MI_RGN_OK)
{
    return s32Result;
}
```

```
}

```

➤ 相关主题

[MI_RGN_Destroy](#)

[MI_RGN_GetAttr](#)

2.1.4 MI_RGN_Destroy

➤ 功能

销毁区域。

➤ 语法

MI_S32 MI_REG_Destroy (MI_U16 u16SocId, [MI_RGN_HANDLE](#) hHandle);

➤ 形参

参数名称	参数含义	输入/输出
s32SocId	芯片 ID，用于级联场景。	输入
hHandle	区域句柄号。 取值范围：[0, MI_RGN_MAX_HANDLE)。	输入

➤ 返回值

返回值 { [MI_RGN_OK](#) 成功。
非 [MI_RGN_OK](#) 失败，参照[返回值](#)。

➤ 依赖

- 头文件：mi_sys.h、mi_rgn.h。
- 库文件：

※ 注意

- 区域必须已创建。

➤ 举例

参见 [MI_RGN_Create](#) 举例。

➤ 相关主题

[MI_RGN_Create](#)

2.1.5 MI_RGN_GetAttr

➤ 功能

获取区域属性。

➤ 语法

MI_S32 MI_RGN_GetAttr(MI_U16 u16SocId, [MI_RGN_HANDLE](#) hHandle, [MI_RGN_Attr_t](#) *pstRegion);

➤ 形参

参数名称	参数含义	输入/输出
s32SocId	芯片 ID，用于级联场景。	输入
hHandle	区域句柄号。 取值范围：[0, MI_RGN_MAX_HANDLE)。	输入
pstRegion	区域属性指针。	输出

➤ 返回值

返回值 { [MI_RGN_OK](#) 成功。
非 [MI_RGN_OK](#) 失败，参照[返回值](#)。

➤ 依赖

- 头文件：mi_sys.h、mi_rgn.h。
- 库文件：

※ 注意

- 区域必须已创建。
- 区域属性指针不能为空。

➤ 举例

参见 [MI_RGN_Create](#) 举例。

➤ 相关主题

无。

2.1.6 MI_RGN_SetBitMap

➤ 功能

设置区域位图，即对区域进行位图填充。

➤ 语法

MI_S32 MI_RGN_SetBitMap(MI_U16 u16SocId, [MI_RGN_HANDLE](#) hHandle,
[MI_RGN_Bitmap_t](#) *pstBitmap);

➤ 形参

参数名称	参数含义	输入/输出
s32SocId	芯片 ID，用于级联场景。	输入
hHandle	区域句柄号。 取值范围：[0, MI_RGN_MAX_HANDLE)。	输入
pstBitmap	位图属性指针。	输入

➤ 返回值

返回值 { [MI_RGN_OK](#) 成功。
非 [MI_RGN_OK](#) 失败，参照[返回值](#)。

➤ 依赖

- 头文件：mi_sys.h、mi_rgn.h。
- 库文件：

※ 注意

- 区域必须已创建。
- 支持位图的大小和区域的大小可以不一致。
- 位图从区域的(0,0)点开始加载。当位图比区域大时，将会自动将图像剪裁成区域大小。
- 位图的像素格式必须和区域的像素格式一致。
- 位图属性指针不能为空。
- 支持多次调用。
- 此接口只对 Overlay 有效。
- 调用了 [MI_RGN_GetCanvasInfo](#) 之后调用本接口无效，除非 [MI_RGN_UpdateCanvas](#) 更新画布生效后。

➤ 举例

```
MI_S32 s32Result = 0;
MI_HANDLE hHandle = 0;
MI_RGN_Bitmap_t stBitmap;
MI_U32 u32FileSize = 200 * 200 * 2;
MI_U8 *pu8FileBuffer = NULL;
FILE *pFile = fopen("200X200.rgb1555", "rb");
if (pFile == NULL)
{
    printf("open file failed \n");
    return -1;
}
pu8FileBuffer = (MI_U8*)malloc(u32FileSize);
if (pu8FileBuffer == NULL)
{
    printf("malloc failed fileSize=%d\n", u32FileSize);
    fclose(pFile);
    return -1;
}
memset(pu8FileBuffer, 0, u32FileSize);
fread(pu8FileBuffer, 1, u32FileSize, pFile);
fclose(pFile);
stBitmap.stSize.u32Width = 200;
stBitmap.stSize.u32Height = 200;
stBitmap.ePixelFormat = E_MI_RGN_PIXEL_FORMAT_RGB1555;
```

```
stBitmap.pData = pu8FileBuffer;
free(pu8FileBuffer);
s32Result = MI_RGN_SetBitMap(0, hHandle, &stBitmap);
```

- 相关主题
无。

2.1.7 MI_RGN_AttachToChn

- 功能

将区域叠加到通道上。

- 语法

```
MI_S32 MI_RGN_AttachToChn(MI_U16 u16SocId, MI_RGN_HANDLE hHandle,
                           MI_RGN_ChnPort_t* pstChnPort,
                           MI_RGN_ChnPortParam_t* pstChnAttr);
```

- 形参

参数名称	参数含义	输入/输出
s32SocId	芯片 ID，用于级联场景。	输入
hHandle	区域句柄号。 取值范围：[0, MI_RGN_MAX_HANDLE]。	输入
pstChnPort	通道端口结构体指针。	输入
pstChnAttr	区域通道显示属性指针。	输入

- 返回值

返回值 { MI_RGN_OK 成功。
非 MI_RGN_OK 失败，参照返回值。

- 依赖

- 头文件：mi_sys.h、mi_rgn.h。
- 库文件：

- ※ 注意

- 区域必须已创建。
- 通道结构体指针不能为空。
- 区域通道显示属性指针不能为空。
- 多个 osd 区域类叠加到同一个通道上，每个 osd 必须是同一个图像格式。
- 并不是所有的通道上都有叠加 region 的能力，下表列出芯片差异。
- COVER 只有硬件 layer。OSD 支持软件 layer，底层会用软件拼图实现。
- OSD、COVER 叠加到通道上对 channel id 不会有要求。

- SCL 进行 rotate 时不支持叠加 OSD 和 COVER。
- 叠加到通道上的 OSD 小于或等于硬件 layer 个数，则全部使用硬件 layer，反之则会用软件拼图。
- 以下表格中所列出的通道位置反映了芯片内部硬件上是否包含 GOP 的硬件模块，并没有具体指明模块的设备 id、通道 id 以及 port id，软件 API 使用者需要结合各模块实际情况来确定实际的属性值，然后通过 [MI_RGN_ChnPort_t](#)* pstChnPort 设定给底层。
 - SCL 对应 MI_SCL 模块，通常在初始化时绑定硬件 SCL 和软件的设备 Id 和 output port id，具体要参考 MI_SCL 模块的 api 文档。
 - DISP 对应 MI_DISP 模块，表格中的 DISPx 与 MI_DISP 模块的设备 id 绑定，只能指定设备 Id，通道 Id 和 port id 需填 0。
 - JPE 和 VENC 对应 MI_VENC 模块的不同 Device（具体需参考 MI_VENC 的 api 文档），通道 Id 根据实际情况填写，port id 对应 MI_VENC 的 input port id。

表 2-1: Tiramisu 芯片信息

芯片名称	通道位置	OSD 各通道支持		Cover Layer 个数	OSD COLOR FORMAT 支持						
		硬件 layer 个数	最大 layer 个数		ARGB1555	ARGB4444	I2	I4	I8	RGB565	ARGB8888
Tiramisu	SCL0	8	128	4	支持	支持	支持	支持	支持	NA	NA
	SCL1	8	128	4	支持	支持	支持	支持	支持	NA	NA
	SCL2	8	128	4	支持	支持	支持	支持	支持	NA	NA
	SCL3	8	128	4	支持	支持	支持	支持	支持	NA	NA
	SCL4	8	128	4	支持	支持	支持	支持	支持	NA	NA
	SCL5	8	128	4	支持	支持	支持	支持	支持	NA	NA

注：此系列芯片内部可用的 GOP 硬件个数为 6 个，可输出的 OSD 的硬件模块有 SCL0/1/2/3/4/5 一共有 6 个。

表 2-2: Muffin 芯片信息

芯片名称	通道位置	OSD 各通道支持		Cover Layer 个数	OSD COLOR FORMAT 支持						
		硬件 layer 个数	最大 layer 个数		ARGB1555	ARGB4444	I2	I4	I8	RGB565	ARGB8888
Muffin	SCL0	8	128	4	支持	支持	支持	支持	支持	NA	NA
	SCL1	8	128	4	支持	支持	支持	支持	支持	NA	NA
	SCL2	8	128	4	支持	支持	支持	支持	支持	NA	NA
	SCL3	8	128	4	支持	支持	支持	支持	支持	NA	NA
	SCL4	8	128	4	支持	支持	支持	支持	支持	NA	NA
	SCL5	8	128	4	支持	支持	支持	支持	支持	NA	NA
	SCL6	8	128	4	支持	支持	支持	支持	支持	NA	NA
	SCL7	8	128	4	支持	支持	支持	支持	支持	NA	NA
	SCL8	8	128	4	支持	支持	支持	支持	支持	NA	NA
	VENC0	8	128	NA	支持	支持	支持	支持	支持	NA	NA
	VENC1	8	128	NA	支持	支持	支持	支持	支持	NA	NA
	JPE0	8	128	NA	支持	支持	支持	支持	支持	NA	NA
	JPE1	8	128	NA	支持	支持	支持	支持	支持	NA	NA

芯片名称	通道位置	OSD 各通道支持		Cover Layer 个数	OSD COLOR FORMAT 支持						
		硬件 layer 个数	最大 layer 个数		ARGB1555	ARGB4444	I2	I4	I8	RGB565	ARGB8888
	DISP0	1	128	NA	支持	支持	支持	支持	支持	NA	NA
	DISP1	1	128	NA	支持	支持	支持	支持	支持	NA	NA
	DISP2	1	128	NA	支持	支持	支持	支持	支持	NA	NA

注：此系列芯片 SCL 内部可用的 GOP 硬件个数为 2 个，可输出的 OSD 的硬件模块有 SCL 0/1/2/3/4/5/6/7/8 一共有 9 个，其中 SCL 0/1/2/3 共用同一个 GOP 硬件，SCL 4/5/6/7/8 共用同一个 GOP 硬件，当多个 SCL 共用一个 GOP 硬件时，同时只能向其中一个 SCL 上贴 OSD。

VENC/JPE/DISP 内部每个 device 都有独立的一个 GOP 硬件。

VENC/JPE OSD 数超过 8 个后不能使用 constant alpha。

表 2-3: Mochi 芯片信息

芯片名称	通道位置	OSD 各通道支持		Cover Layer 个数	OSD COLOR FORMAT 支持						
		硬件 layer 个数	最大 layer 个数		ARGB1555	ARGB4444	I2	I4	I8	RGB565	ARGB8888
Mochi	SCL0	8	128	4	支持	支持	支持	支持	支持	NA	NA
	SCL1	8	128	4	支持	支持	支持	支持	支持	NA	NA
	SCL2	8	128	4	支持	支持	支持	支持	支持	NA	NA
	SCL3	8	128	4	支持	支持	支持	支持	支持	NA	NA
	SCL4	8	128	4	支持	支持	支持	支持	支持	NA	NA
	SCL5	8	128	4	支持	支持	支持	支持	支持	NA	NA
	SCL6	8	128	4	支持	支持	支持	支持	支持	NA	NA
	VENC0	8	128	NA	支持	支持	支持	支持	支持	NA	NA
	JPE0	8	128	NA	支持	支持	支持	支持	支持	NA	NA
	JPE1	8	128	NA	支持	支持	支持	支持	支持	NA	NA
	DISP0	1	128	NA	支持	支持	支持	支持	支持	NA	NA
	DISP1	1	128	NA	支持	支持	支持	支持	支持	NA	NA

注：此系列芯片 SCL 内部可用的 GOP 硬件个数为 1 个，可输出的 OSD 的硬件模块有 SCL 0/1/2/3/4/5/6 一共有 7 个，7 个 SCL 硬件共用同一个 GOP 硬件，当多个 SCL 共用一个 GOP 硬件时，同时只能向其中一个 SCL 上贴 OSD。

VENC/JPE/DISP 内部每个 device 都有独立的一个 GOP 硬件。

VENC/JPE OSD 数超过 8 个后不能使用 constant alpha。

表 2-4: Maruko 芯片信息

芯片名称	通道位置	OSD 各通道支持		Cover Layer 个数	OSD COLOR FORMAT 支持						
		硬件 layer 个数	最大 layer 个数		ARGB1555	ARGB4444	I2	I4	I8	RGB565	ARGB8888
Maruko	SCL0	8	128	4	支持	支持	支持	支持	支持	支持	支持
	SCL1	8	128	4	支持	支持	支持	支持	支持	支持	支持

芯片名称	通道位置	OSD 各通道支持		Cover Layer Layer 个数	OSD COLOR FORMAT 支持						
		硬件 layer 个数	最大 layer 个数		ARGB1555	ARGB4444	I2	I4	I8	RGB565	ARGB8888
	SCL2	8	128	4	支持	支持	支持	支持	支持	支持	支持
	SCL3	8	128	4	支持	支持	支持	支持	支持	支持	支持
	VENC0	8	128	NA	支持	支持	支持	支持	支持	NA	NA
	JPE0	8	128	NA	支持	支持	支持	支持	支持	NA	NA
	DISP0	8	128	NA	支持	支持	支持	支持	支持	支持	支持

注：此系列芯片 SCL 和 DISP 共同使用一个 GOP 硬件，可输出的 OSD 的硬件模块有 SCL 0/1/2/3 和 DISP 0 一共有 5 个，5 个硬件共用同一个 GOP 硬件，当多个模块共用一个 GOP 硬件时，同时只能向其中一个模块上贴 OSD。

VENC/JPE 内部每个 device 都有独立的一个 GOP 硬件。

➤ 举例

```
MI_S32 s32Result = 0;
MI_RGN_HANDLE hHandle = 0;
MI_RGN_ChNPort_t stChNPort;
MI_RGN_ChNPortParam_t stChNAttr;

memset(&stChNPort, 0, sizeof(MI_RGN_ChNPort_t));
memset(&stChNAttr, 0, sizeof(MI_RGN_ChNPortParam_t));
stChNPort.eModId = E_MI_MODULE_ID_SCL;
stChNPort.s32DevId = 0;
stChNPort.s32ChNId = 0;
stChNPort.s32PortId = 0;
stChNAttr.bShow = TRUE;
stChNAttr.stPoint.u32X = 0;
stChNAttr.stPoint.u32Y = 0;
stChNAttr.unPara.stCoverChNPort.u32Layer = 0;
stChNAttr.unPara.stCoverChNPort.stSize.u32Width = 200;
stChNAttr.unPara.stCoverChNPort.stSize.u32Height = 200;
stChNAttr.unPara.stCoverChNPort.u32Color = 0;

s32Result = MI_RGN_AttachToChN(0, hHandle, &stChNPort, &stChNAttr);
if (s32Result != MI_RGN_OK)
{
    return s32Result;
}

s32Result = MI_RGN_DetachFromChN(0, hHandle, &stChNPort);
if (s32Result != MI_RGN_OK)
{
    return s32Result;
}
```

➤ 相关主题

[MI_RGN_DetachFromChN](#)

2.1.8 MI_RGN_DetachFromChn

➤ 功能

将区域从通道中撤出。

➤ 语法

```
MI_S32 MI_RGN_DetachFromChn(MI_U16 u16SocId, MI_RGN_HANDLE hHandle,  
                             MI_RGN_ChnPort_t *pstChnPort);
```

➤ 形参

参数名称	参数含义	输入/输出
s32SocId	芯片 ID，用于级联场景。	输入
hHandle	区域句柄号。 取值范围：[0, MI_RGN_MAX_HANDLE)。	输入
pstChnPort	通道端口结构体指针。	输入

➤ 返回值

返回值 {
MI_RGN_OK 成功。
非 MI_RGN_OK 失败，参照返回值。

➤ 依赖

- 头文件：mi_sys.h、mi_rgn.h。
- 库文件：

※ 注意

- 区域必须已创建。
- 通道结构体指针不能为空。
- 被叠加区域的通道或组（如 VENC，SCL 等）销毁前，需要调用本接口将区域从通道或组中撤出。

➤ 举例

参见 MI_RGN_AttachToChn 举例。

➤ 相关主题

[MI_RGN_AttachToChn](#)

2.1.9 MI_RGN_SetDisplayAttr

➤ 功能

设置区域的通道显示属性。

➤ 语法

```
MI_S32 MI_RGN_SetDisplayAttr(MI_U16 u16SocId, MI_RGN_HANDLE hHandle,  
                              MI_RGN_ChnPort_t *pstChnPort,  
                              MI_RGN_ChnPortParam_t *pstChnPortAttr);
```

➤ 形参

参数名称	参数含义	输入/输出
s32SocId	芯片 ID，用于级联场景。	输入
hHandle	区域句柄号。 取值范围：[0, MI_RGN_MAX_HANDLE)。	输入
pstChnPort	通道端口结构体指针。	输入
pstChnPortAttr	区域通道端口显示属性指针。	输入

➤ 返回值

返回值 { [MI_RGN_OK](#) 成功。
非 [MI_RGN_OK](#) 失败，参照[返回值](#)。

➤ 依赖

- 头文件：mi_sys.h、mi_rgn.h。
- 库文件：

※ 注意

- 区域必须已创建。
- 建议先获取属性，再设置。
- 通道结构体指针不能为空。
- 区域通道显示属性指针不能为空。
- 区域必须先叠加到通道上。

➤ 举例

```
MI_S32 s32Result = 0;
MI_RGN_HANDLE hHandle = 0;
MI_RGN_ChnPort_t stChnPort;
MI_RGN_ChnPortParam_t stChnAttr;

stChnPort.eModId = E_MI_MODULE_ID_SCL;
stChnPort.s32DevId = 0;
stChnPort.s32ChnId = 0;
stChnPort.s32OutputPortId = 0;
s32Result = MI_RGN_GetDisplayAttr(0, hHandle, &stChnPort, &stChnAttr);
if (s32Result != MI_RGN_OK)
{
    return s32Result;
}

stChnAttr.bShow = TRUE;
stChnAttr.stPoint.u32X = 0;
stChnAttr.stPoint.u32Y = 0;
stChnAttr.stCoverPara.u32Layer = 0;
stChnAttr.stCoverPara.stSize.u32Width = 200;
stChnAttr.stCoverPara.stSize.u32Height = 200;
stChnAttr.stCoverPara.u32Color = 0;
```

```
s32Result = MI_RGN_SetDisplayAttr(0, hHandle, &stChnPort, &stChnAttr);
if (s32Result != MI_RGN_OK)
{
    return s32Result;
}
```

➤ 相关主题

[MI_RGN_GetDisplayAttr](#)

2.1.10 MI_RGN_GetDisplayAttr

➤ 功能

获取区域的通道显示属性。

➤ 语法

```
MI_S32 MI_RGN_GetDisplayAttr(MI_U16 u16SocId, MI_RGN_HANDLE hHandle,
                             MI_RGN_ChnPort_t *pstChnPort,
                             MI_RGN_ChnPortParam_t *pstChnPortAttr);
```

➤ 形参

参数名称	参数含义	输入/输出
s32SocId	芯片 ID，用于级联场景。	输入
hHandle	区域句柄号。 取值范围：[0, MI_RGN_MAX_HANDLE)。	输入
pstChnPort	通道端口结构体指针。	输入
pstChnPortAttr	区域通道端口显示属性指针。	输出

➤ 返回值

返回值 { [MI_RGN_OK](#) 成功。
非 [MI_RGN_OK](#) 失败，参照[返回值](#)。

➤ 依赖

- 头文件：mi_sys.h、mi_rgn.h。
- 库文件：

※ 注意

- 区域必须已创建。
- 通道结构体指针不能为空。
- 区域通道显示属性指针不能为空。

➤ 举例

请参见 [MI_RGN_SetDisplayAttr](#) 的举例。

➤ 相关主题

[MI_RGN_SetDisplayAttr](#)

2.1.11 MI_RGN_GetCanvasInfo

➤ 功能

获取区域的显示画布信息。

➤ 语法

```
MI_S32 MI_RGN_GetCanvasInfo(MI_U16 u16SocId, MI\_RGN\_HANDLE hHandle,  
                             MI\_RGN\_CanvasInfo\_t* pstCanvasInfo);
```

➤ 形参

参数名称	参数含义	输入/输出
s32SocId	芯片 ID，用于级联场景。	输入
hHandle	区域句柄号。 取值范围：[0, MI_RGN_MAX_HANDLE)。	输入
pstCanvasInfo	区域显示画布信息。	输出

➤ 返回值

返回值 { [MI_RGN_OK](#) 成功。
非 [MI_RGN_OK](#) 失败，参照[返回值](#)。

➤ 依赖

- 头文件：mi_sys.h、mi_rgn.h。
- 库文件：

※ 注意

- 区域必须已创建。
- 本接口与 [MI_RGN_SetBitMap](#) 功能类似，主要用于 overlay 类型导入位图数据。
- 本接口相对于 [MI_RGN_SetBitMap](#) 接口，用户可以直接更新显示内部画布数据，节省一次内存拷贝和一张画布内存。
- 本接口用于获取区域对应的画布信息，在得到画布地址之后，用户可直接对画布进行操作，譬如：将 bmp 数据直接填写到该画布中。然后通过调用 [MI_RGN_UpdateCanvas](#) 接口，更新显示画布数据。
- 本接口与 [MI_RGN_SetBitMap](#) 接口互斥。如果已经使用了本接口，那么在调用 [MI_RGN_UpdateCanvas](#) 前，调用 [MI_RGN_SetBitMap](#) 不生效。

➤ 举例

```
MI_RGN_HANDLE hHandle;  
MI_RGN_Attr_t stRegion;  
MI_RGN_PaletteTable_t stPaletteTable;  
MI_RGN_CanvasInfo_t stCanvasInfo;  
  
memset(&stPaletteTable, 0, sizeof(MI_RGN_PaletteTable_t));  
stPaletteTable.astElement[0].u8Alpha = 0;  
stPaletteTable.astElement[0].u8Red = 255;  
stPaletteTable.astElement[0].u8Green = 255;
```

```

stPaletteTable.astElement[0].u8Blue = 255;
if (MI_RGN_OK != MI_RGN_Init(0, &stPaletteTable))
{
    printf("Init error!\n");

    return -1;
}
hHandle = 10;
stRegion.eType = E_MI_RGN_TYPE_OSD;
stRegion.stOsdInitParam.ePixelFormat = E_MI_RGN_PIXEL_FORMAT_ARGB1555;
stRegion.stOsdInitParam.stSize.u32Width = 100;
stRegion.stOsdInitParam.stSize.u32Height = 100;
if (MI_RGN_OK != MI_RGN_Create(0, hHandle, &stRegion))
{
    printf("Create handle error!\n");

    return -1;
}
FILE *pFile = fopen("100X100.argb1555", "rb");
if (pFile == NULL)
{
    printf("open file failed \n");
    MI_RGN_Destroy(0, hHandle);

    return -1;
}
if (MI_RGN_GetCanvas(0, hHandle, &stCanvasInfo) != MI_RGN_OK)
{
    return s32Result;
}

for (int i = 0; i < 100; i++)
{
    fread((MI_U8*)stCanvasInfo.virtAddr + i * stCanvasInfo.u32Stride, 1, 100 * 2, pFile);
}
fclose(pFile);
if (MI_RGN_UpdateCanvas(0, hHandle) != MI_RGN_OK)
{
    return s32Result;
}

```

➤ 相关主题

[MI_RGN_UpdateCanvas](#)

2.1.12 MI_RGN_UpdateCanvas

➤ 功能

更新显示画布，若画布有叠加到通道上则更新显示画布，若没有叠加到通道，则在执行叠加操作后才会会在通道上显示出画布的内容。

➤ 语法

MI_S32 MI_RGN_UpdateCanvas(MI_U16 u16SocId, [MI_RGN_HANDLE](#) hHandle);

➤ 形参

参数名称	参数含义	输入/输出
s32SocId	芯片 ID，用于级联场景。	输入
hHandle	区域句柄号。 取值范围：[0, MI_RGN_MAX_HANDLE)。	输入

➤ 返回值

返回值 { [MI_RGN_OK](#) 成功。
非 [MI_RGN_OK](#) 失败，参照[返回值](#)。

➤ 依赖

- 头文件：mi_sys.h、mi_rgn.h。
- 库文件：

※ 注意

- 区域必须已创建。
- 本接口配合 [MI_RGN_GetCanvasInfo](#) 使用。主要用于画布内存数据更新之后，进行画布切换显示。
- 本接口必须与 [MI_RGN_GetCanvasInfo](#) 成对调用，同一个 handler 先执行 [MI_RGN_GetCanvasInfo](#) 获取画布的内存指针，待绘图操作完成后执行本接口，若没有先执行 [MI_RGN_GetCanvasInfo](#)，则会返回 [MI_ERR_RGN_NOT_PERM](#)。
- 当多个 handler 叠加到同一个通道上显示，并且都会使用 [MI_RGN_GetCanvasInfo](#) 及本接口进行绘图操作，在这个通道上的所有绘图操作可以在同一个线程上执行，若在不同线程上，必须使用锁保护起来，否则有可能会出现问题 OSD 闪烁，或者 osd 不刷新的问题。
- 以上多线程的情况下锁的使用详细说明如下：
rgn 的 handle 假设有 handler0 和 handler1 都贴到同一个通道上
分别两个线程
handler0
T0_0 = getcanvas
T0_1 = update
handler1
T1_0 = getcanvas
T1_1 = update
按照时间 T 的顺序执行
有问题的情况：
T0_0 -> T1_0 -> T0_1 -> T1_1，并伴随底层 error 的打印：“Front buf state error!!!”。
加 mutex 后，能够正常。调整后的时序：
lock -> T0_0 -> T0_1 -> unlock -> lock -> T1_0 -> T1_1 -> unlock

➤ 举例

请参见 [MI_RGN_GetCanvasInfo](#) 的举例。

➤ 相关主题

[MI_RGN_GetCanvasInfo](#)

2.1.13 MI_RGN_InitDev

➤ 功能

初始化 RGN 设备。

➤ 语法

```
MI_S32 MI_RGN_InitDev(MI_U16 u16SocId, MI\_RGN\_InitParam\_t *pstInitParam);
```

➤ 形参

参数名称	参数含义	输入/输出
s32SocId	芯片 ID，用于级联场景。	输入
pstInitParam	设备初始化参数	输入

➤ 返回值

返回值 { [MI_RGN_OK](#) 成功。
非 [MI_RGN_OK](#) 失败，参照[返回值](#)。

➤ 依赖

- 头文件：mi_sys.h、mi_rgn.h。
- 库文件

※ 注意

- 此接口用于替换原有 [MI_RGN_Init](#) 接口。

➤ 举例

无。

➤ 相关主题

无。

2.1.14 MI_RGN_DeInitDev

➤ 功能

反初始化 RGN 设备。

➤ 语法

```
MI_S32 MI_RGN_DeInitDev(MI_U16 u16SocId);
```

➤ 形参

参数名称	参数含义	输入/输出
s32SocId	芯片 ID，用于级联场景。	输入

➤ 返回值

返回值 { [MI_RGN_OK](#) 成功。

非 [MI_RGN_OK](#) 失败，参照[返回值](#)。

➤ 依赖

- 头文件：mi_sys.h、mi_rgn.h。
- 库文件

※ 注意

- 此接口用于替换原有 [MI_RGN_DeInit](#) 接口。

➤ 举例

无。

➤ 相关主题

无。

3. RGN 数据类型

视频前处理相关数据类型、数据结构定义如下：

数据类型	定义
MI_RGN_MAX_HANDLE	定义区域的最大句柄数
MI_RGN_MAX_PALETTE_TABLE_NUM	颜色表最大元素个数
MI_RGN_HANDLE	定义区域句柄
MI_RGN_Type_e	定义区域类型
MI_RGN_PixelFormat_e	RGB or Index 格式
MI_RGN_AlphaMode_e	设定 Osd Alpha 显示模式
MI_RGN_Size_t	大小信息
MI_RGN_OsdAlphaAttr_t	Osd Alpha 属性的结构体
MI_RGN_OsdInitParam_t	定义 Osd 区域属性结构体
MI_RGN_PaletteElement_t	定义颜色元素
MI_RGN_PaletteTable_t	定义颜色表
MI_RGN_Attr_t	定义区域类型结构体
MI_RGN_Bitmap_t	定义位图图像信息结构
MI_RGN_ChnPort_t	定义模块设备通道结构体
MI_RGN_Point_t	定义坐标信息结构体
MI_RGN_CoverChnPortParam_t	定义遮挡区域的通道显示属性
MI_RGN_OsdChnPortParam_t	定义 OSD 区域的通道显示属性
MI_RGN_OsdArgb1555Alpha_t	Argb1555 格式的前景、背景 Alpha 设定
MI_RGN_ChnPortParamUnion_u	定义 Region 属性的联合体
MI_RGN_AlphaModePara_u	定义 OSD Alpha Mode 参数联合体
MI_RGN_ChnPortParam_t	定义区域通道显示属性结构体
MI_RGN_CanvasInfo_t	定义画布信息结构体
MI_RGN_InitParam_t	定义 RGN 设备初始化参数

3.1. MI_RGN_MAX_HANDLE

- 说明
定义区域的最大句柄。
- 定义
`#define MI_RGN_MAX_HANDLE 1024`
- 成员
无
- ※ 注意事项
无
- 相关数据类型及接口
无

3.2. MI_RGN_MAX_PALETTE_TABLE_NUM

- 说明
颜色表最大元素个数。
- 定义
`#define MI_RGN_MAX_PALETTE_TABLE_NUM 256`
- 成员
无
- ※ 注意事项
无
- 相关数据类型及接口
无

3.3. MI_RGN_HANDLE

- 说明
定义区域句柄。
- 定义
`typedef MI_U32 MI_RGN_HANDLE;`

➤ 成员

成员名称	描述
MI_RGN_HANDLE	区域句柄。

※ 注意事项
无

➤ 相关数据类型及接口
无

3.4. MI_RGN_Type_e

➤ 说明

定义区域类型。

➤ 定义

```
typedef enum
{
    E_MI_RGN_TYPE_OSD = 0,
    E_MI_RGN_TYPE_COVER,
    E_MI_RGN_TYPE_MAX
} MI_RGN_Type_e;
```

➤ 成员

成员名称	描述
E_MI_REG_OSD	视频叠加区域。
E_MI_REG_COVER	视频遮挡区域。

※ 注意事项
无

➤ 相关数据类型及接口
无

3.5. MI_RGN_PixelFormat_e

➤ 说明

定义叠加区域属性结构体。

➤ 定义

```
typedef enum
{
    E_MI_RGN_PIXEL_FORMAT_ARGB1555 = 0,
    E_MI_RGN_PIXEL_FORMAT_ARGB4444,
    E_MI_RGN_PIXEL_FORMAT_I2,
```

```

E_MI_RGN_PIXEL_FORMAT_I4,
E_MI_RGN_PIXEL_FORMAT_I8,
E_MI_RGN_PIXEL_FORMAT_RGB565,
E_MI_RGN_PIXEL_FORMAT_ARGB8888,
E_MI_RGN_PIXEL_FORMAT_MAX
} MI_RGN_PixelFormat_e;

```

➤ 成员

成员名称	描述
E_MI_RGN_PIXEL_FORMAT_ARGB1555	ARGB1555 格式
E_MI_RGN_PIXEL_FORMAT_ARGB4444	ARGB4444 格式
E_MI_RGN_PIXEL_FORMAT_RGBI2	I2 格式（两个 bit 表示，支援 4 种颜色，调色板查色）
E_MI_RGN_PIXEL_FORMAT_RGBI4	I4 格式（4 个 bit 表示，支援 16 种颜色，调色板查色）
E_MI_RGN_PIXEL_FORMAT_I8	I4 格式（8 个 bit 表示，支援 256 种颜色，调色板查色）
E_MI_RGN_PIXEL_FORMAT_RGB565	RGB565
E_MI_RGN_PIXEL_FORMAT_ARGB8888	ARGB8888 格式

※ 注意事项

每个 chip 支持的图像格式不一样，API 中极大化地罗列出了所有的图像格式，但是有些格式存在 API 不支援的情况，如果使用者需要 chip 支援的情况，请查看 region procfs。

使用命令：echo getcap > /proc/mi_modules/mi_rgn/mi_rgn0。

使用者无法设定 index 0 的调色盘，I2/I4/I8 这些格式的 index 0 被底层 driver 用作 color key，表示这种颜色不被硬件识别，所以当全 0 的数据叠加到通道上时，是不显示任何颜色的。

Color key 的数值可以在 procfs 的 getcap 中查看，colorkey 的数值是一个 16bit 整型，它的高 8 位和低 8 位是一样的值，当使用 Index 类型的 colorformat 时，对内存数据进行 memset 0 即可让硬件不识别，当使用 RGB 或者 ARGB 格式时无论使用的是何种排列，对内存数据进行 memeset (colorkey & 0xFF) 数值即可。

➤ 相关数据类型及接口

无

3.6. MI_RGN_AlphaMode_e

➤ 说明

设定 Osd Alpha 显示模式。

➤ 定义

```

typedef enum
{
    E_MI_RGN_PIXEL_ALPHA = 0,
    E_MI_RGN_CONSTANT_ALPHA

```

```
}MI_RGN_AlphaMode_e;
```

➤ 成员

成员名称	描述
E_MI_RGN_PIXEL_ALPHA	Osd 显示每个 pixel 对应的 alpha 效果，例如 argb1555/argb4444/argb8888/i2/i4/i8 这些格式都能支持 pixel alpha，rgb565 则不会生效。
E_MI_RGN_CONSTANT_ALPHA	Osd 硬件会忽略图像格式中的 alpha 位，使用统一的值设定 alpha 值。例如 rgb565 能设定其透明度。

※ 注意事项
无

➤ 相关数据类型及接口
无

3.7. MI_RGN_Size_t

➤ 说明

定义大小信息结构体。

➤ 定义

```
typedef struct MI_RGN_Size_s  
{  
    MI_U32 u32Width;  
    MI_U32 u32Height;  
} MI_RGN_Size_t;
```

➤ 成员

成员名称	描述
u32Width	宽度
u32Height	高度

※ 注意事项
无

➤ 相关数据类型及接口
无

3.8. MI_RGN_OsdAlphaAttr_t

➤ 说明

定义 Osd Alpha 属性的结构体。

➤ 定义


```
typedef struct MI_RGN_OsdAlphaAttr_s
{
    MI\_RGN\_AlphaMode\_e eAlphaMode;
    MI\_RGN\_AlphaModePara\_u stAlphaPara;
}MI_RGN_OsdAlphaAttr_t;
```

➤ 成员

成员名称	描述
eAlphaMode	Osd Alpha 的使用模式。
stAlphaPara	模式对应的参数。

※ 注意事项
无

➤ 相关数据类型及接口
无

3.9. MI_RGN_OsdInitParam_t

➤ 说明

定义叠加区域属性结构体。

➤ 定义

```
typedef struct MI_RGN_OsdInitParam_s
{
    MI\_RGN\_PixelFormat\_e ePixelFormat;
    MI\_RGN\_Size\_t stSize;
}MI_RGN_OsdInitParam_t;
```

➤ 成员

成员名称	描述
ePixelFormat	像素格式。
stSize	区域的宽高。 宽最大不超过 3840 高最大不超过 2160。

※ 注意事项
ePixelFormat、stSize 只在调用 [MI_RGN_AttachToChn](#) 后，[MI_RGN_DetachFromChn](#) 之前为静态变量。

➤ 相关数据类型及接口
无

3.10. MI_RGN_PaletteElement_t

➤ 说明

定义颜色元素。

➤ 定义

```
typedef struct MI_RGN_PaletteElement_s
{
    MI_U8 u8Alpha;
    MI_U8 u8Red;
    MI_U8 u8Green;
    MI_U8 u8Blue;
}MI_RGN_PaletteElement_t;
```

➤ 成员

成员名称	描述
u8Alpha	透明度
u8Red	红色
u8Green	绿色
u8Blue	蓝色

※ 注意事项

无

➤ 相关数据类型及接口

无

3.11. MI_RGN_PaletteTable_t

➤ 说明

定义颜色表。

➤ 定义

```
typedef struct MI_RGN_PaletteTable_s
{
    MI\_RGN\_PaletteElement\_t astElement[MI_RGN_MAX_PALETTE_TABLE_NUM];
}MI_RGN_PaletteTable_t;
```

➤ 成员

成员名称	描述
astElement	颜色元素

※ 注意事项

无

- 相关数据类型及接口

无

3.12. MI_RGN_Attr_t

- 说明

定义区域属性结构体。

- 定义

```
typedef struct MI_RGN_Attr_s
{
    MI\_RGN\_Type\_e eType;
    MI\_RGN\_OsdInitParam\_t stOsdInitParam;
}MI_RGN_Attr_t;
```

- 成员

成员名称	描述
eType	区域类型。
stOsdInitParam	osd 区域属性。

- ※ 注意事项

无

- 相关数据类型及接口

无

3.13. MI_RGN_Bitmap_t

- 说明

定义位图图像信息结构。

- 定义

```
typedef struct MI_RGN_Bitmap_s
{
    MI\_RGN\_PixelFormat\_e ePixelFormat;
    MI\_RGN\_Size\_t stSize;
    void *pData;
} MI_RGN_Bitmap_t;
```

- 成员

成员名称	描述
------	----

ePixelFormat	位图像素格式
stSize	位图宽度，高度，stride
pData	位图数据

※ 注意事项
无

➤ 相关数据类型及接口
无

3.14. MI_RGN_ChnPort_t

➤ 说明
定义模块设备通道结构体。

➤ 定义

```
typedef struct MI_RGN_ChnPort_s
{
    MI_ModuleId_e eModId;
    MI_S32 s32DevId;
    MI_S32 s32ChnId;
    MI_S32 s32PortId;
}MI_RGN_ChnPort_t;
```

➤ 成员

成员名称	描述
eModId	模块号
s32DevId	设备号
s32ChnId	通道号
s32PortId	输出端口 id

※ 注意事项
无

➤ 相关数据类型及接口
无

3.15. MI_RGN_Point_t

➤ 说明
定义坐标信息结构体。

➤ 定义

```
typedef struct MI_RGN_Point_s
{
    MI_U32 u32X;
    MI_U32 u32Y;
}MI_RGN_Point_t;
```

➤ 成员

成员名称	描述
u32X	横坐标
u32Y	纵坐标

※ 注意事项
无

➤ 相关数据类型及接口
无

3.16. MI_RGN_CoverChnPortParam_t

➤ 说明

定义遮挡区域的通道显示属性。

➤ 定义

```
typedef struct MI_RGN_CoverChnPortParam_s
{
    MI_U32 u32Layer;
    MI_RGN_Size_t stSize;
    MI_U32 u32Color;
}MI_RGN_CoverChnPortParam_t;
```

➤ 成员

成员名称	描述
u32Layer	区域层次，数值低的在底层
stSize	Cover 宽，高
u32Color	颜色，VYU444

※ 注意事项

- Cover 的宽高不是绝对宽高，而是基于最大值为 8192 的相对值。
如：Cover 叠加到的目标通道宽高为 1920x1080，要设定宽高为 960x540 的 Cover，此处的 stSize 要设定为(4096, 4096)。
- u32Color 代表 Cover 的显示颜色，其中 23~16 位代表 V 分量，15~8 位代表 Y 分量，7~0 代表 U 分量。

- 相关数据类型及接口
无

3.17. MI_RGN_OsdChnPortParam_t

- 说明
定义 OSD 区域的通道显示属性。
- 定义

```
typedef struct MI_RGN_OsdChnPortParam_s
{
    MI_U32 u32Layer;
    MI\_RGN\_OsdAlphaAttr\_t stOsdAlphaAttr;
}MI_RGN_OsdChnPortParam_t;
```
- 成员

成员名称	描述
u32Layer	区域层次，数值低的在底层
stOsdAlphaAttr	Osd Alpha 属性

- ※ 注意事项
无
- 相关数据类型及接口
无

3.18. MI_RGN_OsdArgb1555Alpha_t

- 说明
Argb1555 格式的前景、背景 Alpha 设定。
- 定义

```
typedef struct MI_RGN_OsdArgb1555Alpha_s
{
    MI_U8 u8BgAlpha;
    MI_U8 u8FgAlpha;
}MI_RGN_OsdArgb1555Alpha_t;
```
- 成员

成员名称	描述
u8BgAlpha	背景 Alpha，Alpha bit 为 0 时对应的 Alpha 值，取值范围 0~0xFF。
u8FgAlpha	前景 Alpha，Alpha bit 为 1 时对应的 Alpha 值，取

	值范围 0~0xFF。
--	-------------

※ 注意事项
无

➤ 相关数据类型及接口
无

3.19. MI_RGN_ChnPortParamUnion_u

➤ 说明
定义 Region 属性的联合体。

➤ 定义

```
typedef union
{
    MI\_RGN\_CoverChnPortParam\_t stCoverChnPort;
    MI\_RGN\_OsdChnPortParam\_t stOsdChnPort;
} MI_RGN_ChnPortParamUnion_u;
```

➤ 成员

成员名称	描述
stCoverChnPort	遮挡区域在通道上的属性设置
stOsdChnPort	Osd 区域在通道上的属性设置

※ 注意事项
无

➤ 相关数据类型及接口
无

3.20. MI_RGN_AlphaModePara_u

➤ 说明
定义 Osd Alpha Mode 参数联合体。

➤ 定义

```
typedef union
{
    MI\_RGN\_OsdArgb1555Alpha\_t stArgb1555Alpha;
    MI_U8 u8ConstantAlpha;
} MI_RGN_AlphaModePara_u;
```

➤ 成员

成员名称	描述
------	----

stArgb1555Alpha	Pixel alpha 时 Argb1555 格式的前景 Alpha 及背景 Alpha 设定。
u8ConstantAlpha	Constant Alpha 时 Alpha 值设定，取值范围 0~0xFF。

※ 注意事项
无

➤ 相关数据类型及接口
无

3.21. MI_RGN_ChnPortParam_t

➤ 说明
定义区域通道显示属性结构体。

➤ 定义

```
typedef struct MI_RGN_ChnPortParam_s
{
    MI_BOOL bShow;
    MI_RGN_Point_t stPoint;
    MI_RGN_ChnPortParamUnion_u unPara;
} MI_RGN_ChnPortParam_t;
```

➤ 成员

成员名称	描述
bShow	区域是否显示。 取值范围：MI_TRUE 或者 MI_FALSE。 动态属性。
stPoint	区域起始点坐标。 当 RGN 类型为 OSD 时，要求 X 坐标按 1byte 对齐。
unPara	Region 通道显示属性。

※ 注意事项
无

➤ 相关数据类型及接口
无

3.22. MI_RGN_CanvasInfo_t

➤ 说明
定义画布信息结构体。

➤ 定义

```
typedef struct MI_RGN_CanvasInfo_s
{
    MI_PHY phyAddr;
    MI_VIRT virtAddr;
    MI\_RGN\_Size\_t stSize;
    MI_U32 u32Stride;
    MI\_RGN\_PixelFormat\_e ePixelFormat;
} MI_RGN_CanvasInfo_t;
```

➤ 成员

成员名称	描述
phyAddr	画布物理地址。
virtAddr	画布虚拟地址
stSize	画布尺寸
u32Stride	画布的 stride
ePixelFormat	画布的像素格式

※ 注意事项

无

➤ 相关数据类型及接口

无

3.23. [MI_RGN_InitParam_t](#)

➤ 说明

定义 RGN 设备初始化参数。

➤ 定义

```
typedef struct MI_RGN_InitParam_s
{
    MI\_RGN\_PaletteTable\_t *pstPaletteTable;
} MI_RGN_InitParam_t;
```

➤ 成员

成员名称	描述
pstPaletteTable	颜色元素指针

※ 注意事项

无

➤ 相关数据类型及接口

MI_RGN_InitDev

4. 返回值

区域管理 API 返回值如下表所示。

错误码	宏定义	描述
0	MI_RGN_OK	成功
0xA0030000	MI_NOTICE_RGN_BUFFER_CHANGE	Buffer 发生改变。当设置属性时会发生，需要重新 remap.
0xA0032001	MI_ERR_RGN_INVALID_HANDLE	非法的句柄
0xA0032001	MI_ERR_RGN_INVALID_DEVID	设备 ID 超出合法范围
0xA0032002	MI_ERR_RGN_INVALID_CHNID	通道组号错误或无效区域句柄
0xA0032003	MI_ERR_RGN_ILLEGAL_PARAM	参数超出合法范围
0xA0032004	MI_ERR_RGN_EXIST	重复创建已存在的设备、通道或资源
0xA0032005	MI_ERR_RGN_UNEXIST	试图使用或者销毁不存在的设备、通道或者资源
0xA0032006	MI_ERR_RGN_NULL_PTR	函数参数中有空指针
0xA0032007	MI_ERR_RGN_NOT_CONFIG	模块没有配置
0xA0032008	MI_ERR_RGN_NOT_SUPPORT	不支持的参数或者功能
0xA0032009	MI_ERR_RGN_NOT_PERM	该操作不允许，如试图修改静态配置参数
0xA003200C	MI_ERR_RGN_NOMEM	分配内存失败，如系统内存不足
0xA003200D	MI_ERR_RGN_NOBUF	分配缓存失败，如申请的数据缓冲区太大
0xA003200E	MI_ERR_RGN_BUF_EMPTY	缓冲区中无数据
0xA003200F	MI_ERR_RGN_BUF_FULL	缓冲区中数据满
0xA0032010	MI_ERR_RGN_NOTREADY	系统没有初始化或没有加载相应模块
0xA0032011	MI_ERR_RGN_BADADDR	地址非法
0xA0032012	MI_ERR_RGN_BUSY	系统忙

5. PROCFS 介绍

5.1. cat

- 调试信息

cat /proc/mi_modules/mi_rgn0

```
----- Start dump region capability info -----
Module name : REGION

Limitation :
Osd supported format : fmt    0 Support
Osd supported format : fmt    1 Support
Osd supported format : fmt    2 Support
Osd supported format : fmt    3 Support
Osd supported format : fmt    4 Support
Osd supported format : fmt    5 Unsupport
Osd supported format : fmt    6 Unsupport
Region handle : 1024 (0 ~ 1023)
channel       : 64 (0 ~ 63)
Osd attach    : 128 (each channel port)
Cover attach  : 4 (each channel port)

Osd support :
Width  : 1 ~ 4096
Height : 1 ~ 4096
HW mode : 8 frontbuffs
Color key value : 0x23232323
X pos overlap : Support
Overlap : Support

Cover support :
Size :
Width  : 1 ~ 8192
Height : 1 ~ 8192
Overlap : Support
-----End dump region capability info -----
```

```
----- Start dump region attr -----
Handle   Type      Width  Height  Stride  Format  PhyAddr  VirAddr  CanvasCnt  UsingCnt
128      COVER
129      COVER
130      COVER
131      COVER
0        OSD      240    135    480     0       60ca4000  0        1          1
1        OSD      240    135    480     0       60cb4000  0        1          1
2        OSD      240    135    480     0       60cc4000  0        1          1
3        OSD      240    135    480     0       60cd4000  0        1          1
4        OSD      240    135    480     0       60ce4000  0        1          1
5        OSD      240    135    480     0       60cf4000  0        1          1
6        OSD      240    135    480     0       60d04000  0        1          1
7        OSD      240    135    480     0       60d14000  0        1          1
----- End dump region attr -----
```

```

----- Start dump Mod 34 device 1 Channel 0 port 0 info -----
----- ChnPort Info -----
CanvasWidth  CanvasHeight  ScreenWidth  ScreenHeight  BlitTimeouCnt  RealtimeFlip
      1920           1080      1920           1080              0          Disable
----- frontbuff info -----
Index bShow  OffsetX  OffsetY  Width  Height  Stride  Format  VirAddr  PhyAddr  AlphaMode  AlphaVal  BgAlpha  FgAlpha  BufCnt  OsdCnt
0      1    1680      0    240    135    480      0      0 60d14000  Pixel      255    255      0      1
1      1    1440      0    240    135    480      0      0 60d04000  Pixel      255    255      0      1
2      1    1200      0    240    135    480      0      0 60cf4000  Pixel      255    255      0      1
3      1     960      0    240    135    480      0      0 60ce4000  Pixel      255    255      0      1
4      1     720      0    240    135    480      0      0 60cd4000  Pixel      255    255      0      1
5      1     480      0    240    135    480      0      0 60cc4000  Pixel      255    255      0      1
6      1     240      0    240    135    480      0      0 60cb4000  Pixel      255    255      0      1
7      1      0      0    240    135    480      0      0 60ca4000  Pixel      255    255      0      1
----- region info -----
Handle  Type  bShow  Layer  Color  Width  Height  Stride  PositionX  PositionY  Format  VirAddr  PhyAddr  AlphaMode  BgAlpha  FgAlpha
128  COVER  1    128  4cff4c54  1024  1024      0      0      0      0      0      0      0      Pixel  255    255
129  COVER  1    129  71e4713f  1024  1024      0      7168     7168      0      0      0      0      Pixel  255    255
130  COVER  1    130  97c9972a  1024  1024      0      4096     4096      0      0      0      0      Pixel  255    255
131  COVER  1    131  bcdfbc15  1024  1024      0      7680     7680      0      0      0      0      Pixel  255    255
0    OSD  1      0      0      240    135      0      0      0      0      0      0      0      Pixel  255    255
1    OSD  1      0      0      240    135      0      240     0      0      0      0      0      Pixel  255    255
2    OSD  1      0      0      240    135      0      480     0      0      0      0      0      Pixel  255    255
3    OSD  1      0      0      240    135      0      720     0      0      0      0      0      Pixel  255    255
4    OSD  1      0      0      240    135      0      960     0      0      0      0      0      Pixel  255    255
5    OSD  1      0      0      240    135      0     1200     0      0      0      0      0      Pixel  255    255
6    OSD  1      0      0      240    135      0     1440     0      0      0      0      0      Pixel  255    255
7    OSD  1      0      0      240    135      0     1680     0      0      0      0      0      Pixel  255    255
----- End dump Mod 34 Dev 1 Channel 0 port 0 info -----
----- Start dump info of buffers in used -----
Memory used:
kmmalloc : 6840 bytes
mma malloc : 518400 bytes
----- End dump info of buffers in used -----

```

➤ 调试信息分析

记录当前 RGN 的使用状况以及 device 属性可以动态地获取到这些信息，方便调试和测试

➤ 参数说明

参数		描述
Region capability	Region handle	创建 region 的最大数量为 1024 个，句柄取值为[0~1023]
	channel	最大通道数量为 64 个，通道取值为[0~63]
	Osd attach	每个输出端口每个通道能绑定的 Osd 数量上限为 128 个
	Cover attach	每个输出端口每个通道能绑定的 Cover 数量上限为 4 个
	Overlay Supported Format	Osd 支持的颜色格式： 0: ARGB1555，宽度按 2 个像素对齐 1: ARGB4444，宽度按 2 个像素对齐 2: I2，宽度按 4 个像素对齐 3: I4，宽度按 2 个像素对齐 4: I8 5: RGB565 6: ARGB8888
	Osd support	Width : Osd 宽取值范围为 1 ~ 4096 Height : Osd 高取值范围为 1~ 4096

参数		描述
		Color key value: 系统默认的 colorkey 的值，除了 index 的格式，不管 osd 的 format 是多少，只要内存中的值是此就表示 colorkey X pos overlap: 支持多个 osd 在 x 方向上重叠。 Overlap: osd 是否支持重叠 HW mode: Osd 的 frontbuff 数量，目前设置为 8
	Cover support	Width : Osd 宽取值范围为 1 ~ 8192 Height : Osd 高取值范围为 1~ 8192 Overlap: Cover 是否支持重叠

参数		描述
Region attr	Handle	句柄
	Type	类型，Osd 或是 Cover
	Width	宽
	Height	高
	Stride	宽补齐
	Format	颜色格式 [0-6], ARGB1555, ARGB4444, I2, I4, I8, RGB565, ARGB8888
	VirAddr	Canvas 的虚拟地址
	PhyAddr	Canvas 的物理地址
	CanvasCnt	当前 Canvas 使用的 buffer 数量
	UsingCnt	使用当前 Canvas 的 hw 数量

参数			描述
Channel Port Info	Frontbuffer info	Index	Frontbuffer 索引
		bShow	是否显示
		OffsetX	X 偏移
		OffsetY	Y 偏移
		Width	宽
		Height	高
		Stride	宽补齐

参数			描述
		Format	颜色格式
		VirAddr	虚拟地址
		PhyAddr	物理地址
		AlphaMode	Alpha 模式，指 Pixel alpha 还是 constant alpha
		AlphaVal	如果是 constant alpha，alpha 的值
		BgAlpha	如果是 pixel alpha，argb1555 的背景 alpha
		FgAlpha	如果是 pixel alpha，argb1555 的前景 alpha
		BufCnt	通道上 front buffer 的个数
		OsdCnt	通道上一个 front buffer 由多少个 back buffer 拼接而成
	Attach info	Handle	绑定 Region 句柄
		Type	绑定 Region 类型
		bShow	是否显示
		Layer	绑定 Cover 的层级
		Color	绑定 Cover 的颜色值
		Width	宽
		Height	高
		Stride	宽补齐
		PositionX	X 偏移
		PositionY	Y 偏移
		Format	绑定 Osd 的颜色格式
		VirAddr	虚拟地址
		PhyAddr	物理地址
		AlphaVal	如果是 constant alpha，alpha 的值
		BgAlpha	如果是 pixel alpha，argb1555 的背景 alpha
		FgAlpha	如果是 pixel alpha，argb1555 的前景 alpha

参数		描述
Buffer info	kmalloc	Region 模块内部申请内存字节数。
	mma alloc	Region 模块通过 Sys 模块申请内存字节数。

5.2. echo

功能	Dump 指定 Region 的 buffer
命令	echo dumpRgnBuf [Handle] [Path] > /proc/mi_modules/mi_rgn/mi_rgn0
参数说明	[Handle] region 句柄

	[Path] 保存 dump 数据的路径。保存内容为 region 的 canvas 内容
举例	echo dumpRgnBuf 0 /mnt > /proc/mi_modules/mi_rgn/mi_rgn0 在/mnt 下产生文件 Rgn0_canvasInfo_fmt0_64X48 表示 dump 句柄为 0，颜色格式为 ARGB1555，Stride 为 64，高为 48 的 region 的 canvas 数据。 文件格式：Rgn[Handle]_canvasInfo_fmt[Format]_[Stride]X[Height] [Handle] region 句柄 [Format] 颜色格式，ARGB1555 为 0，ARGB4444 为 1，I2 为 2，I4 为 3 [Stride] 宽补齐 [Height] 高

功能	Dump 指定 channel 和 port 的 frontbuffer
命令	echo dumpFrontBuf [ModId] [DevId] [ChnID] [PortID] [Path] > /proc/mi_modules/mi_rgn/mi_rgn0
参数说明	[ModId] 模块号，参考 mi_common_datatype.h 中的模块定义枚举值。
	[DevId] 模块的设备号
	[ChnID] 通道号 [0 ~ 63]
	[PortID] 端口号 [0~3]
	[Path] 保存 dump 数据的路径。会根据当前实际使用的 frontbuffer 数量生成 0 ~ 2 个文件
举例	echo dumpFrontBuf 34 1 0 0 /mnt > /proc/mi_modules/mi_rgn/mi_rgn0 在/mnt 下产生 Mod34_Dev1_Ch0_Port0_frontbuf0_fmt0_64X48, 表示 dump 通道为 0，输出端口为 34 号端口，索引为 0 的 frontbuffer 的数据，颜色格式为 ARGB1555，Stride 为 64，高为 48。 文件格式： Mod[ModId]_Dev[DevId]_Chn[Channel]_Port[Port]_frontbuf[Index]_fmt[Format]_[Stride]X[Height] [ModId] 输出模块号 [DevId] 输出模块的设备号 [Channel] 通道号 [Port] 输出端口 [Index] frontbuffer 索引号 [Format] 颜色格式，ARGB1555 为 0，ARGB4444 为 1，I2 为 2，I4 为 3 [Stride] 宽补齐 [Height] 高

功能	获取 region 能力级信息
命令	echo getcap > /proc/mi_modules/mi_rgn/mi_rgn0
参数说明	无
举例	无

功能	获取已创建 region 信息
命令	echo dumprgn > /proc/mi_modules/mi_rgn/mi_rgn0
参数说明	无
举例	无

功能	获取 channel 和 port 的信息
命令	echo dumpchport > /proc/mi_modules/mi_rgn/mi_rgn0
参数说明	无
举例	无

功能	获取内存使用信息
命令	echo bufcnt > /proc/mi_modules/mi_rgn/mi_rgn0
参数说明	无
举例	无

功能	获取 Palette 信息
命令	echo dumpPalette > /proc/mi_modules/mi_rgn/mi_rgn0
参数说明	无
举例	无

功能	开关某个 gwin
命令	echo setDispOnOff [ModId] [DevId] [ChnID] [PortID] [Idx] [OnOff] > /proc/mi_modules/mi_rgn/mi_rgn0
参数说明	[ModId] 模块号，参考 mi_common_datatype.h 中的模块定义枚举值。
	[DevId] 模块的设备号
	[ChnID] 通道号 [0 ~ 63]
	[PortID] 端口号 [0~3]
	[Idx] 当前通道上 osd 显示 gwin 的序号。
	[OnOff] 是否开关，0 表示关、1 表示开。
举例	echo setDispOnOff 34 1 0 0 0 0 > /proc/mi_modules/mi_rgn/mi_rgn0

功能	强制更新 osd 的 back buffer
命令	echo forceUpdate [handle] > /proc/mi_modules/mi_rgn/mi_rgn0
参数说明	[handle] Osd handler
举例	echo forceUpdate 0 > /proc/mi_modules/mi_rgn/mi_rgn0

功能	设置显示 front buffer 的个数
命令	echo setFbListMaxCnt [ModId] [DevId] [ChnID] [PortID] [SetByUsr] [Cnt] > /proc/mi_modules/mi_rgn/mi_rgn0
参数说明	[ModId] 模块号，参考 mi_common_datatype.h 中的模块定义枚举值。 [DevId] 模块的设备号 [ChnID] 通道号 [0 ~ 63] [PortID] 端口号 [0~3] [SetByUsr] 是否通过手动设置，1:是，0: 否，系统默认是 0； [Cnt] 若是手动设置，则 front buffer 用于显示的个数，此设定设置完后会在 Frontbuffer info 的 BufCnt 中查看到
举例	echo setFbListMaxCnt 34 1 0 0 1 3 > /proc/mi_modules/mi_rgn/mi_rgn0

功能	使能 OSD 实时显示模式，确保在用户操作后的下一帧视频画面即可显示 OSD
命令	echo setRealtimeFlip [ModId] [DevId] [ChnID] [PortID] [OnOff] > /proc/mi_modules/mi_rgn/mi_rgn0
参数说明	[ModId] 模块号，参考 mi_common_datatype.h 中的模块定义枚举值。 [DevId] 模块的设备号 [ChnID] 通道号 [0 ~ 63] [PortID] 端口号 [0~3] [OnOff] RealtimeFlip 使能状态
举例	echo setRealtimeFlip 34 1 0 0 1 >/proc/mi_modules/mi_rgn/mi_rgn0

功能	限制指定 handle 最大使用 buffer 数。
命令	echo setMaxCanvasForHandle [handleId] [LimitNum] > /proc/mi_modules/mi_rgn/mi_rgn0
参数说明	[handleId] 句柄号 [LimitNum] 设定的最大 buffer 数。
举例	echo setMaxCanvasForHandle 0 2 > /proc/mi_modules/mi_rgn/mi_rgn0

功能	限制指定 channel 和 port 的最大使用 buffer 数
命令	echo setMaxCanvasForPath [ModId] [DevId] [ChnID] [PortID] [LimitNum] > /proc/mi_modules/mi_rgn/mi_rgn0
参数说明	[ModId] 模块号，参考 mi_common_datatype.h 中的模块定义枚举值。 [DevId] 模块的设备号 [ChnID] 通道号 [0 ~ 63] [PortID] 端口号 [0~3] [LimitNum] 设定的最大 buffer 数。
举例	echo setFbListMaxCnt 34 1 0 0 2 > /proc/mi_modules/mi_rgn/mi_rgn0